

Assignment 8: Introduction to React

The purpose of this assignment is to learn the basics of React by creating a simple client-side application. The application will allow the user to get hilarious jokes to brighten their day! The jokes will come from a third-party API called the Official Joke API; check out its documentation here: https://github.com/15Dkatz/official_joke_api

The jokes from this API seem to be pretty inoffensive, but I haven't read all of them so I can't guarantee it – use at your own risk and let me know if you see anything objectionable or if you'd prefer to use a different API.

In the starter files, I've included a video of how the application should work. I've also included a stylesheet that you can use if you want, although it's not necessary.

Task 1: Setup

In your project folder, set up two subfolders—**express** and **react**—that each have a separate Node.js application. One application should run an Express application serving static files, and the other should run **Parcel**, compiling the React bundle to the **express** folder to be sent to the client when requested. You can use one of the demo applications from the slides or video lessons as a starting point.

Task 2: App Hierarchy and Type Chooser

This application will eventually have at least three components, but for now, let's start with two. Create a new component called **App**; as you may expect, this will be the parent component that contains all other components in the application.

Next, create a component called **TypeChooser** and have it rendered in **App**. The purpose of this component is to render a list of different types of jokes that a user can choose from.

In the **TypeChooser** component, create an array of types with the **useState** hook within the **TypeChooser** component.

The joke types are the following:

```
'general',  
'knock-knock',  
'programming',  
'dad'
```

Have the component render this array as an unordered list with anchor tags. Remember that each list item must have a **key**. (The types will always be uniquely named, so you can simply use the type names for the key.)

The hyperlinks don't have to do anything when you click on them yet. That's coming up in the next task.

Task 2: Joke Component

Now create a new component called **Joke** and have it rendered in the **App** component. The purpose of this component is to display a joke that corresponds to the chosen type. But before requesting a joke from the API, let's first just have this component render the **type** of joke that was chosen in the **TypeChooser** component. Note that **TypeChooser** and **Joke** are sibling components that must now share information with each other.

Use an event listener function to detect when a type has been clicked. The chosen type should be stored in the state somewhere (figure out the best place for this!) and then rendered in the **Joke** component. Use **event.preventDefault()** to prevent the anchor tag from reloading the page when clicked.

A few notes:

- Remember that sibling components cannot communicate directly with one another. They must communicate through a parent.
- Remember to use the appropriate state setter function to update the state.

Task 3: Loading the Joke from Official Joke API

You should now have a simple three-component application that allows one to choose a joke type from the **TypeChooser** component and see the type rendered in the **Joke** component. Now we need to request a joke from Official Joke API that corresponds to the chosen type.

Use the **useEffect** hook to make an asynchronous GET request to the Official Joke API whenever **Joke** component receives an updated **type** prop. Use the API documentation to figure out how to get a random joke corresponding to one particular type:

https://github.com/15Dkatz/official_joke_api

Once the joke has been loaded, it should be saved in state. Figure out which component should store the joke data. Render the **set up** and **punchline** in the **Joke** component.

At this point, all basic functionality of the application should be working: the user should be able to click on a type in the **TypeChooser** component and see a joke displayed in the **Joke** component. Next, we'll add a few extra features and functionalities.

Task 4: Loading a New Joke

Our application currently provides a randomized joke each time a new **type** is chosen. But what if the user wants to get a new joke of the currently-chosen type? Simply clicking on the type again in **TypeChooser** won't do it, because the **Joke** component only loads a new joke when a new type has been received as a prop.

There are a few ways to handle this, but what I'd like you to do is add a button to the **Joke** component with text **"Get another joke"** that loads a new joke of the currently-chosen type when clicked. This button should not appear in the application until *after* the first time a type has been chosen; after all, it doesn't make sense to load a new joke if a type hasn't yet been chosen yet!

Task 5: Loading Indicator

Requesting a new joke from the API is an **asynchronous** action, and there's a chance that it may take longer than expected to get a response. In the case that a response is delayed, we should let the user know that their choice has at least been received and a response is forthcoming. It may be confusing and frustrating for the user if they choose a type and then nothing seems to happen. So, let's add a text indicator that says "loading..." to let the user know when the application is waiting for a response from the API. **Figure out how to do this.** A few notes:

- The "loading..." message should not appear until *after* a type has been chosen. (so not when the application is first loaded.)
- Setter methods for changing the state are also asynchronous. So there is actually more than one asynchronous thing happening here, and we need to make sure that all asynchronous operations have completed before removing the loading message. Hint: use the **useEffect** hook to check to make sure the necessary state update has occurred before removing the loading message.
- The Official Joke API is actually quite fast, so it may be difficult to test this functionality. To artificially slow down your connection, open the **inspector**, navigate to the **Network** tab, click on the dropdown that says "**No throttling**" and choose a slower connection speed. (This will cause the application to load glacially slow when you refresh the page, so you may have to frequently switch back to No throttling.)

Task 6: Highlight Chosen Type

The last thing to do is implement a small style enhancement to improve the usability of the application: change the background color of the currently-chosen type in the **TypeChooser** component. To do this, use a JavaScript style object applied within the component itself. Figure out how to do this. (See the demo video, in which I used **lightblue** as the background color.)

Hand In

Delete the **.node_modules** folder and **.parcel-cache** in both the **react** and **express** subfolders, rename the project folder to **a8-firstname-lastname**, zip the folder, and hand it in to D2L.

Grading

- [1 mark] Basic React application setup
- [1 mark] TypeChooser component lists all types
- [1 mark] State updated when type chosen
- [2 marks] Joke loaded from API upon choosing type
- [1 mark] "Get another joke" button implemented
- [2 marks] Loading message displayed after new type chosen but before new joke loaded
- [1 mark] Chosen type is colored in TypeChooser component

total: 9

As usual, up to -25% could be deducted for improper hand in, disorganized project, etc.
Please ask me if in doubt.